Comparative Analysis of Q-Learning and DQN for Intelligent Taxi Dispatching

Dogyun Kwon *Iroo Jeon **Chanhee Jung, ***Jiyeong Hwang
Sungkyunkwan University

Abstract

Efficient vehicle dispatching is a critical component of modern Intelligent Transportation Systems, requiring intelligent sequential decision-making to optimize long-term performance. This study presents a comparative analysis of two foundational Reinforcement Learning (RL) algorithms—Tabular Q-Learning and Deep Q-Networks (DQN)—applied to a taxi dispatching problem. By formulating the task as a Markov Decision Process within the Taxi-v3 environment, we evaluate the trade-offs between computational simplicity and state generalization. In the standard 5×5 grid, Tabular Q-Learning serves as a robust baseline, achieving a perfect success ratio with greater training stability and lower computation cost than DQN. However, scalability experiments expanding the grid to 15×15 demonstrate the critical limitations of tabular methods as the state space increases. In the same scenarios, DQN exhibits more graceful degradation, maintaining a measurably better reward profile due to the advantages of neural function approximation. Ultimately, this research confirms a scale-dependent view: while Q-Learning is highly practical for small discrete tasks, DQN provides a more promising direction for scaling toward complex, realistic dispatching systems.

1. Introduction

Efficient vehicle dispatching is an important problem in modern Intelligent Transportation Systems (ITS). Conventional greedy strategies, such as assigning the nearest available taxi. They are simple and intuitive but often ignore future passenger demand and vehicle positioning. Therefore, taxi dispatching requires to improve long-term cumulative performance. This problem can be naturally formulated as a sequential decision-making task. Each action taken by a taxi affects later states and rewards. Reinforcement Learning (RL) is suitable for this because an agent can learn an optimal policy through repeated interaction with the environment.

In this study, we compare two value-based RL algorithms: Tabular Q-Learning and Deep Q-Networks (DQN). Using the Gymnasium Taxi-v3 environment, we analyze their learning performance, stability, and scalability. In particular, we examine how the methods work on expanding grid, focusing on the trade-off between the simplicity of Q-Learning and the generalization ability of DQN.

The main contributions of this study are as follows:

1. We formulate the taxi dispatching problem as a Markov Decision Process and compare Q-Learning and DQN under the same environment.
2. We analyze how state-space expansion affects performance, showing the scalability limitation of tabular methods and the potential advantage of neural function approximation.

2. Related Works

RL has long been studied for vehicle taxi dispatching. Dietterich [1] introduced the Taxi environment as a benchmark for hierarchical RL and demonstrated how complex tasks can be decomposed into smaller subtasks. This work provided an important foundation for modeling taxi dispatching as a Markov Decision Process (MDP), although it mainly focused on small discrete environments where tabular methods are feasible.

Later, Mnih et al. [2] introduced Deep Q-Networks (DQN), which use neural networks to approximate Q-values in large or high-dimensional state spaces. By combining experience replay and target networks, DQN improved the stability of deep RL and made it possible to apply value-based methods beyond small tabular problems.

Recent studies have extended these ideas to more realistic taxi dispatching scenarios. Zhou et al. [3] applied robust deep reinforcement learning to driverless taxi dispatching under uncertain demand, while Rajeh et al. [4] proposed a multi-agent reinforcement learning framework for finer-grained city-scale taxi coordination. Compared with these advanced settings, our study focuses on a controlled benchmark environment to clearly compare the strengths and limitations of Q-Learning and DQN as the state space increases.

3. Method

We formalize the taxi dispatching problem as an MDP, represented by a tuple $(S, A, P, R\ \gamma)$.

3.1 State Space

In the standard 5×5 benchmark, the state space S is a discrete set which consists of 500 possible states, derived from the product of the followings: 1. Taxi Coordinates, a grid

of 25 locations; 2. Passenger Location, 5 states (destinations and 1 state representing the passenger being inside the vehicle); 3. Destination, 4 designated landmarks $\{R, G, Y, B\}$. To evaluate scalability, we extend this to $N \times N$. As N increases, the state space grows quadratically, specifically $|S| = N^2 \times 5 \times 4$. This expansion is utilized to test the limits of the Q-table versus the generalization capabilities of the DQN's neural network.

3.2 Action Space & Reward function

The action space A consists of 6 discrete actions that the agent can execute at each time step:

- Movement: South (0), North (1), East (2), West (3)
- Operational: Pickup (4), Drop-off (5)

The reward function $R(s, a)$ is designed as +20 for successfully delivering passenger at a destination, -10 for wrongly attempted Pickup or Drop-off, and -1 for all other actions. The step-wise penalty -1 is critical for encouraging the agent to find the shortest path, thereby minimizing idle cruising time.

3.3 Q-Learning Method

Q-Learning was used as a tabular baseline. Since Taxi-v3 has 500 states and 6 actions, a Q-table was initialized as a 500 × 6 matrix of zeros. Each entry $Q(s, a)$ represents the expected long-term return of taking action a in state s. The Q-values were updated using the standard temporal-difference rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max Q(s', a') - Q(s, a)]$$

Action selection followed an ε-greedy policy. The exploration rate started from $\varepsilon = 1.0$, decayed after each episode. This allowed sufficient exploration in the early stage and more exploitation in the later stage. Additionally, two practical improvements were added. First, random tie-breaking was used when multiple actions had the same maximum Q-value. Second, the best Q-table was saved based on the highest 100-episode moving average reward, instead of using only the final Q-table.

To analyze the learning process in more detailed ways, we recorded the episode reward, episode length, and 100-episode moving average reward throughout training. These records were later used for performance visualization and comparison with DQN.

3.4 DQN Method

Unlike Q-learning, DQN approximates the action-value function $Q(s,a)$ using neural networks, allowing generalization across larger states spaces. The current state was encoded as a one-hot vector and passed into the Q-network, which outputs Q-values for all 6 actions. The Q-network consisted of two hidden fully connected layers with ReLU activations. Actions were selected using an ε-greedy policy. Each transition (s, a, r, s') was stored in a replay buffer, and random mini-batches were sampled for training. The experience replay reduces sample correlation, improving data efficiency.

To stabilize training, we used a separate target network in addition to the online Q-network. The target value for each sampled transition was computed as

$$y = r + \gamma * \max_{a'} Q_target(s', a').$$

The online Q-network was trained by minimizing the mean-squared error between the predicted Q-value and this target. The target network was periodically updated from the online network, reducing instability caused by changing targets.

During training, we recorded episode reward, episode length, and the 100-episode moving average reward. For evaluation, the learned policy was tested with greedy action selection, and average reward and average episode length were measured.

4. Data and Experiment

All experiments were conducted using the Taxi-v3 environment. To improve re-producibility, random seeds were fixed with deterministic seed offsets applied to each episode. We trained a basic Q-learning model and an improved version with random tie-breaking and best-model check-pointing. The main hyperparameters were set as follows: learning rate $\alpha = 0.1$, discount factor $\gamma = 0.99$ and maximum episode length of 200 steps. The model was trained for 8,000 episodes. The basic model was evaluated over 100 episodes using a greedy policy and the improved model was over 200 episodes using the best Q-table. When multiple actions had the same maximum Q-value, one action was selected uniformly at random.

We used four evaluation metrics: average episode reward, average number of steps, success ratio, and high-quality episode ratio. The success ratio was defined as the proportion of episodes with non-negative total reward, while the high-quality episode ratio was defined as the proportion of episodes with reward greater than or equal to 8. We also visualized training reward, moving average reward, episode length, and evaluation reward/step distributions.

5. Results

5.1 Q-Learning Result

The Q-learning agent showed a clear improvement during training. In the early stage, rewards were strongly negative because the agent frequently made inefficient moves and invalid pickup/drop-off actions. As exploration decreased, the policy became more structured, and the 100-episode moving average reward steadily improved. In the improved implementation, the moving average rose from -394.82 at episode 500 to 7.75 by episode 6,000, with the best recorded moving average reward of 8.23, as shown in Fig. 1.

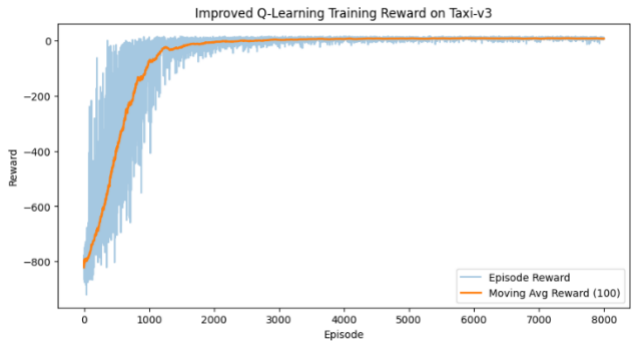

Fig. 1 Improved Q-Learning Training Reward on Taxi-v3

The basic Q-learning model already produced a strong result, with an average evaluation reward of 7.63 and an average episode length of 13.37 over 100 evaluation episodes. This suggests that, in a small discrete environment such as Taxi-v3, a tabular method can learn an effective policy without function approximation.

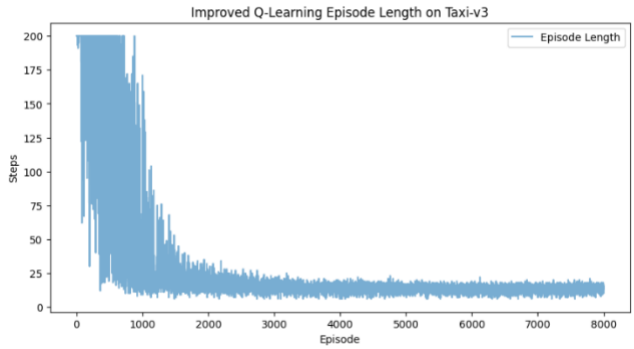

Fig. 2 Improved Q-Learning Episode Length on Taxi-v3

The improved Q-learning model performed slightly better and more consistently. Over 200 evaluation episodes, it achieved an average reward of 7.96, an average episode length of 13.04 steps, a success ratio of 1.000, and a high-quality episode ratio of 0.520. These results show that the added improvements not only raised performance slightly, but also made the learned policy more stable across repeated evaluations. Another meaningful result is that episode length decreased as learning progressed. Since given -1 penalty, it indicates that the agent learned not only to complete the task, but also to reduce wasted actions. This trend can be observed in Fig. 2, where the number of steps becomes lower and more stable in the later stage of training.

The evaluation reward distribution also supports this interpretation. As shown in Fig. 3, many evaluation episodes are concentrated in a favorable reward range, suggesting that the learned policy was reliable as well as high-performing on average.

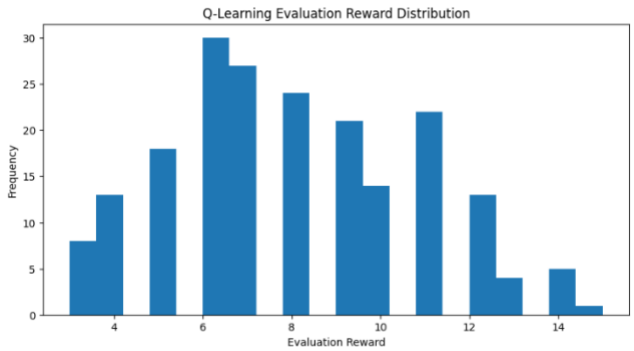

Fig. 3 Q-Learning Evaluation Reward Distribution

Overall, the Q-learning results show that tabular reinforcement learning remains a strong baseline in small, fully discrete dispatching environments. Its implementation is simple, training is stable after sufficient exploration, and the final policy achieves both high reward and efficient task completion. However, this strength depends on the state space being limited and explicitly enumerable.

5.2 DQN Result

The DQN agent also showed a clear learning trend in the Taxi-v3 environment. At the beginning of training, episode rewards were strongly negative and unstable because the agent explored frequently and often received step penalties or invalid-action penalties. However, as training continued, the moving-average reward increased steadily, indicating that the Q-network gradually learned more useful state-action values.

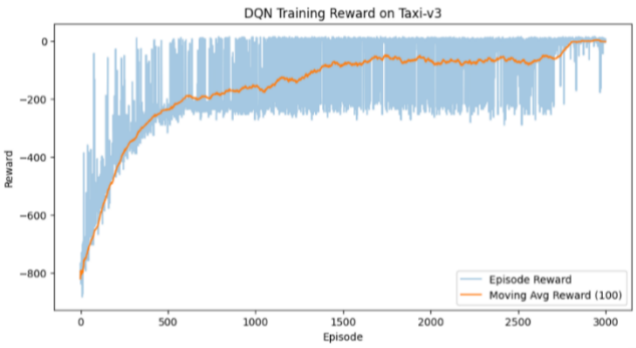

Fig. 4 DQN Training Reward on Taxi-v3

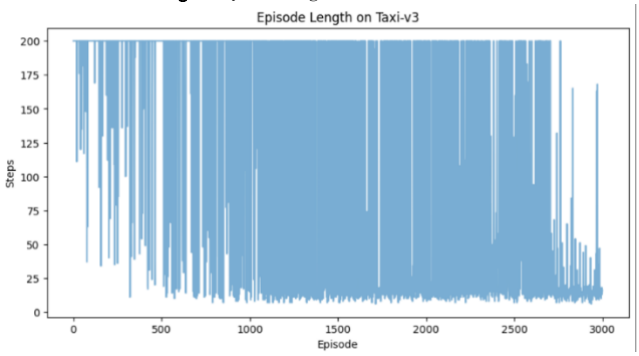

Fig. 5 DQN Episode Length on Taxi-v3

In the reward curve of the DQN agent, although the raw episode rewards remain noisy, which is common in deep RL, the moving-average line rises from a poor initial level to a much better range near the end of training. This suggests that

the agent gradually moved from random exploration to more structured decision making. In the early stage, many episodes reached the maximum step limit, meaning that the agent often failed to solve the task efficiently. As learning progressed, shorter episodes appeared more frequently, implying that the agent learned to complete pickup-and-drop-off sequences with fewer unnecessary moves. Although the step curve still fluctuates, the overall pattern supports the reward result and shows improved efficiency over time.

5.3 Comparison between Q-Learning and DQN

In the standard environment, Q-learning is a strong baseline because the state space is small and fully discrete. It can directly store and update all state-action values without approximation error, leading to fast convergence and stable behavior. By contrast, the advantage of DQN is partly offset by higher computation cost, relative instability in learning and greater sensitivity to hyperparameters. Therefore, Q-learning is more practical for the original Taxi-v3 benchmark, while DQN becomes more meaningful when the environment grows larger. In this sense, Q-learning is suitable for small discrete dispatching tasks, whereas DQN provides a more scalable direction for larger taxi dispatching problems.

6. Conclusion

This study evaluated the comparative performance of Tabular Q-Learning and DQN in a taxi dispatching task. In small-scale environments, Q-Learning served as a robust and efficient baseline, offering superior training stability and lower computational cost. However, as the state space expanded, the fundamental limitations of tabular methods were revealed , whereas DQN maintained strong performance by leveraging state generalization through neural function approximation. In conclusion, while Q-Learning is practical for simple, low-dimensional tasks, DQN provides a more promising and scalable framework for complex, high-dimensional real-world dispatching scenarios. Future research should extend to multi-agent environments and stochastic demand patterns to better reflect the complexities of city-scale logistics.

Reference

[1] T. G. Dietterich, “Hierarchical Reinforcement Learning with the MAXQ Value Function Decomposition,” arXiv.org. Accessed: Apr. 20, 2026. [Online]. Available: https://arxiv.org/abs/cs/9905014v1.

[2] V. Mnih et al., “Human-level control through deep reinforcement learning,” Nature, vol. 518, no. 7540, pp. 529–533, Feb. 2015, doi: 10.1038/nature14236.

[3] X. Zhou, L. Wu, Y. Zhang, Z.-S. Chen, and S. Jiang, “A robust deep reinforcement learning approach to driverless taxi dispatching under uncertain demand,” Information Sciences, vol. 646, p. 119401, Oct. 2023, doi: 10.1016/j.ins.2023.119401.

[4] T. M. Rajeh, Z. Luo, M. H. Javed, F. Alhaek, and T. Li, “A Clustering-Based Multi-Agent Reinforcement Learning